Machine Intelligence Research
www.mi-research.net
DOI: 10.1007/s11633-025-1579-8

A New Meta-path Monte Carlo Tree Search Algorithm for Heterogeneous Graph Neural Networks

Bin Zhang[1] Zhipeng Cheng[1] Chenglong Du[2] Zhengganzhe Chen[2]
Chaoyang Chen[3] Weihua Gui[2]

[1]School of Automation, Guangdong University of Technology, Guangzhou 510006, China
[2]School of Automation, Central South University, Changsha 410083, China
[3]School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan 411201, China

Abstract: Heterogeneous graph neural networks (HGNN) can capture heterogeneous semantic information in heterogeneous networks, learn the low-dimensional embedding vectors, and use them for downstream tasks. The selection of meta-paths is always the focus of HGNN. Existing HGNN models often employ random selections of meta-paths or utilize all meta-paths with a fixed maximum number of hops, thereby overlooking significant heterogeneous semantic information of graphs and struggling to effectively leverage non-redundant information. To this end, a new Monte Carlo tree search-based heterogeneous graph neural network (MCTS-HGNN) model is developed to search for the appropriate set of meta-paths in heterogeneous graphs automatically, thus overcoming the difficulty of meta-path selection. Subsequently, the meta-path set is decomposed based on aggregation objects and independently applied to a subset of meta-paths by using a customized transformer-based semantic aggregation module, and then the diverse semantic information from meta-paths can be effectively utilized. Furthermore, the information from the meta-path subset is integrated by the graph-level transformer to achieve a comprehensive heterogeneous graph embedding. The learned embedding is evaluated via the downstream task of the heterogeneous graph. Finally, the ablation experiments validate the effectiveness of the module designed for the MCTS-HGNN. The experimental results demonstrate that the MCTS-HGNN outperforms state-of-the-art baselines across all evaluation metrics.

Keywords: Graph neural network, heterogeneous network, meta-path search, node embedding, node classification.

Citation: B. Zhang, Z. Cheng, C. Du, Z. Chen, C. Chen, W. Gui. A new meta-path Monte Carlo tree search algorithm for heterogeneous graph neural networks. *Machine Intelligence Research*. http://doi.org/10.1007/s11633-025-1579-8

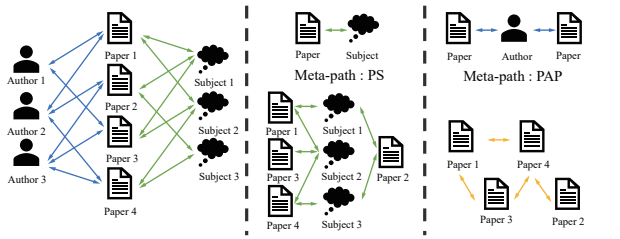
1 Introduction

The network structure is almost everywhere in the real world, such as social networks, recommendation systems, and biological information. Nodes represent individuals or organizations, and edges between nodes indicate relationships between entities. Graph neural networks (GNNs) have achieved significant performance in graph related tasks[1-5], including graph attribute prediction, link prediction, and node classification, etc. Most GNNs are designed for homogeneous graphs consisting of a single node type and a single relationship type between nodes. However, heterogeneous graphs with multiple node types and multiple relationship types are commonly found in the real world[6]. Meanwhile, various heterogeneous graph neural networks (HGNNs) have been proposed to solve different tasks in heterogeneous graphs[7-10].

The rich semantics of heterogeneous graphs make their modeling difficult. As shown in Fig. 1 of the citation network, academic graphs typically contain three node types (paper, author, and subject), as well as three relationships (write, quote and belongs to), for example, author → writes → paper. Due to the semantic differences in the relationships between nodes in heterogeneous graphs, learning node embeddings in such networks is often difficult[11]. Although traditional GNNs have performed well in homogeneous graph tasks, they cannot handle the complex semantic relationships of different structures in heterogeneous graphs. As a result, HGNNs have received increasing attention from scholars in recent years.

A meta-path is a composite relationship path between two objects, and is typically used to represent multiple types of semantic relationships[11]. HGNNs can be divided into two categories: meta-path-free methods and meta-path-based methods. Most meta-path-free methods

Research Article
Manuscript received on January 25, 2025; accepted on July 3, 2025
Recommended by Associate Editor Hao Sun
Colored figures are available in the online version at https://link.springer.com/journal/11633
© Institute of Automation, Chinese Academy of Sciences and Springer-Verlag GmbH Germany, part of Springer Nature 2026



(a) Heterogeneous graph (b) Connection graphs based on PS (c) Connection graphs based on PAP

Fig. 1 Citation network and meta-path

are based on graph convolutional networks (GCNs)[1] and combine attention mechanisms or matrix factorization methods to extend graph convolution to heterogeneous graphs, such as relational graph convolutional networks (RGCN)[12], heterogeneous graph structural attention neural network (HetSANN)[13], explanation-aware graph convolution network (ExpGCN)[14], relation-aware HGNN (R-HGNN)[15], etc. Most of the above methods based on meta-paths focus on node aggregation and semantic fusion from heterogeneous graphs, in order to capture rich semantic information from heterogeneous graphs, such as heterogeneous graph attention network (HAN)[7], heterogeneous graph propagation network (HPN)[9], heterogeneous graph neural network (HetGNN)[16], simple and efficient HGNN (SeHGNN)[17], etc.

Most existing HGNN models based on meta-paths require manual selection of meta-paths or select all meta-paths through a fixed maximum hop count, resulting in issues such as loss of information or introduction of other noise in the original dataset. However, identifying valuable meta-paths can be formulated as a search problem. Assuming that the node type set size is $|\mathcal{A}|$, the relationship type size is $|\mathcal{R}|$, and the fixed maximum number of hops is l, the potential meta-paths reside in a search space of size $(|\mathcal{A}| \cdot |\mathcal{R}|)^l$. This enormous search space can lead to a combinatorial explosion.

Previous research has shown that meta-path influences the model performance in a negative or positive way[18]. However, most existing semantic aggregation methods ignore the problems brought by the characteristics of meta-paths themselves, such as their symmetry and subgraph heterogeneity. Without classification, all meta-paths are aggregated at once, resulting in problems such as semantic loss or redundancy, as well as low utilization of semantic aggregation information between meta-paths.

To address these problems, the main contributions of this paper are as follows:

1) A novel Monte Carlo tree-based meta-path search framework is developed, enabling effective searching of meta-paths through leveraging diverse adjacent information for improved embedding.

2) A new attention-based method for aggregating different meta-paths is proposed, which achieves the calculation of meta-paths involving similar and dissimilar types of aggregate objects, thereby improving the learning of semantic information in graphs.

3) Furthermore, the information from the meta-path subset is integrated by the graph-level transformer to achieve a comprehensive heterogeneous graph embedding.

4) Experiments on the citation network dataset (association for computing machinery (ACM)), bibliographic website dataset (digital bibliography & library project (DBLP)), and movie network dataset (internet movie database (IMDB)) show that the developed model is superior to the state-of-the-art baselines in node classification, node clustering, and visualization tasks. The ablation study demonstrates that the proposed meta-path search framework and meta-path aggregation method significantly improve the performance of the model.

2 Related work

2.1 GNN

The GNN was first proposed by Scarselli et al.[19], which uses recurrent neural networks as the main framework and generates embeddings for each node through feature mapping and node aggregation. Kipf and Welling[1] adopted a first-order approximation based on Chebyshev convolution and proposed GCN to further simplify the graph convolution process, but with stronger expressive power. GraphSAGE[20] performs neural network-based aggregation on fixed size node neighbors, obtaining feature information by aggregating neighboring information. Graph attention network (GAT)[2] uses attention to learn the importance of nodes and their neighboring nodes, assigning different weights to different nodes in the neighborhood, without the need for any expensive matrix operations. Homophily degree estimation and label utilization-GNN (HLA-GNN)[21] integrates homophily degree estimation and label utilization into message passing, enhancing the quality of node representation learning in low-homogeneity graphs. It achieves label propagation enhancement and high-order information aggregation, while also demonstrating superior representation learning capabilities in high-homogeneity graphs. Regarding the critical topological aspects of graphs, Xiong et al.[22] proposed a novel fusion model named autoencoder and graph neural network (AGNN), which integrates the structural details of networks into graph embeddings by combining autoencoder and GNN architectures, thereby enhancing the model's effectiveness.

2.2 HGNNs

The current HGNNs can be divided into two categories: meta-path-free methods and meta-path-based methods.

The meta-path-free method: RGCN[12] extends GCN to heterogeneous graphs by fusing different relationships separately and then overlaying them. HetSANN[13] maps

different types of nodes to multiple low dimensional spaces, and then aggregates various relationship information between nodes using GNN and attention mechanisms. ExpGCN[14] divides subgraphs based on the types of edges in heterogeneous graphs, and generates node representations by aggregating information from different subgraphs. R-HGNN[15] decomposes the graph into relational graphs using convolutional learning nodes, and finally adapts the aggregated information based on weighted residual connections.

The meta-path-based method: HAN[7] uses node level and semantic level attention, while considering the importance of nodes and meta-paths. SeHGNN[17] uses a lightweight mean aggregator to calculate neighbor aggregation based on HAN, reducing complexity by removing overly used neighbor attention and avoiding repeated neighbor aggregation during each training period. Long-range meta-path search through progressive sampling (LMSPS)[23] studied the importance of different meta-paths and conducted remote meta-paths search through progressive sampling. Regarding the self-supervised learning of heterogeneous networks, SESIM[24] constructs supervision information by predicting the number of hops between nodes in each meta-path, which is used as a pretext task to improve the performance of the primary task.

2.3 Meta-path search strategy

Currently, the meta-paths in existing HGNNs are almost always manually selected. Most real-world heterogeneous graphs are large and complex, and as the size and complexity grow, it becomes increasingly difficult to select reasonable meta-paths. Shi and Weninger[25] used improved classical knowledge discovery techniques to mine useful meta-paths from complex heterogeneous graphs, and introduced how to use meta-paths for node similarity measurement and path search. Meng et al.[26] proposed a forward stage path generation (FSPG) algorithm that uses greedy algorithms to select the most relevant meta-paths. Wan et al.[27] proposed a framework for meta-path reasoning that uses a reinforcement learning agent to generate effective meta-paths in large-scale heterogeneous graphs. Genetic meta-structure search (GEMS)[28] uses parallel genetic algorithms to search for meaningful meta-structures for recommendation, leveraging dedicated rules and meta-structure predictors to effectively explore the search space. Ning et al.[18] proposed a reinforcement learning-based meta-path selection (RMS) framework, which uses a specific policy model to generate a meta-path set, and designs a recommendation model for the RMS framework.

The above work has made great contributions to the research of meta-path search, but searching for meaningful meta-structures on large-scale heterogeneous graphs remains an extraordinary problem. Reinforcement learning-based models such as RMS-Hrec and GEMS typically require substantial search time. In contrast, LMSPS employs progressive random sampling to achieve superior results compared to reinforcement learning models within shorter timeframes. However, while the LMSPS gradually narrows the meta-path set through mean-value sampling, it still faces significant challenges in determining the optimal search direction. Research on how to search for suitable meta-paths significantly helps improve model performance.

3 Preliminary

To more clearly describe our method, this section introduces related definitions of HGNN, including heterogeneous graph, meta-path, meta-path tree, and heterogeneous graph embedding.

Definition 1[29]. A heterogeneous graph is defined as $G = \{V, E, \mathcal{A}, \mathcal{R}\}$, where V is the node set, E is the edge set, $\mathcal{A}$ is the set of node types, and $\mathcal{R}$ is the set of edge types. There exist mapping functions $\phi : V \to \mathcal{A}$ and $\psi : E \to \mathcal{R}$ such that for each node $v_i \in V$, $\phi(v_i) \in \mathcal{A}$, and for each edge $e_i \in E$, $\psi(e_i) \in \mathcal{R}$, with $|\mathcal{A}| + |\mathcal{R}| > 2$.

Definition 2[11]. A meta-path $\mathcal{P}$ is a composite relation that consists of multiple edge types, for example, $\mathcal{P} \triangleq c_1 \leftarrow c_2 \leftarrow \cdots \leftarrow c_l$ ($\mathcal{P} = c_1 c_2 \cdots c_l$ for short), where $c_i \in \mathcal{A}$.

Definition 3. Given a heterogeneous graph $G = \{V, E, \mathcal{A}, \mathcal{R}\}$, and a meta-path set Φ, then the meta-path tree can be expressed as $T_\Phi = \{Q, W, f_Q, f_W\}$, where

1) Q: Tree node set, this set is composed of node types in the heterogeneous graph;

2) W: Tree edge set, which is used to connect parent and child nodes;

3) f_Q: Tree node mapping function $f_Q : Q \to \Phi$, which uniquely maps tree nodes to meta-paths, for example $f_Q(q_i) = \mathcal{P}_i$;

4) f_W: Tree node edge weight mapping function $f_W : W \to \Phi_A$, which maps the edges of the parent node q_i and child node q_j of the tree to the heterogeneous graph adjacency matrix $\hat{A}_{qiqj}$ between node types q_i and q_j, for example $f_W(q_i, q_j) = \hat{A}_{qiqj}$.

4 Methodology

In this section, a novel Monte Carlo tree search-based heterogeneous graph neural network model (MCTS-HGNN) is proposed, where a new stage called meta-path search is designed for more effectively capturing heterogeneous semantic information of meta-path. The model comprises three modules: 1) Meta-path search, where a meta-path tree construction method and an MCTS-based search strategy are designed to explore optimal paths; 2) node aggregation, which performs node aggregation and feature projection under selected meta-paths via adjacency matrices to generate meta-path embeddings; 3) semantic aggregation, which leverages a transformer to hierarchically fuse semantics after decomposing meta-path sets. The overall framework is illustrated in Fig. 2,

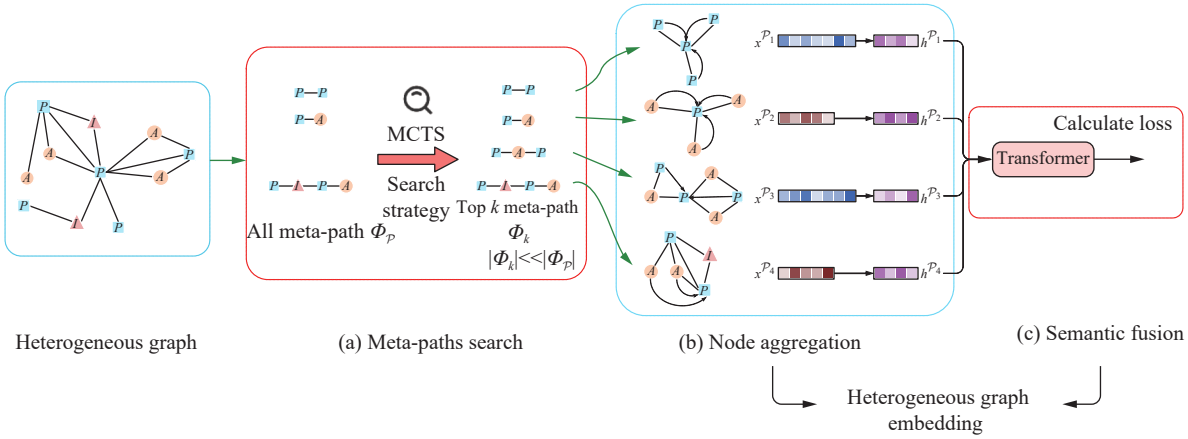


Fig. 2 Heterogeneous graph neural network based on Monte Carlo tree search for meta-path (Colored figures are available in the online version at https://link.springer.com/journal/11633)

with the detailed algorithm provided in Algorithms 1 and 2.

4.1 Improved meta-path search strategy

Due to the characteristics of meta-path structure, the number of meta-paths increases exponentially with the growth of hops. For simplification, the meta-path set can be represented as a tree structure. For example, meta-path set $\Phi = \{PP, PA, PI, PPP, PPA, PPI, PIA, PII, PPAP, \cdots\}$, the corresponding meta-path tree $T_\Phi = \{Q, W, f_Q, f_W\}$ is shown in Fig. 3, where node set $Q = \{P_1, P_2, P_3, P_4, A_1, A_2, A_3, I_1, I_2, I_3, \cdots\}$ and edge set $W = \{P_1P_2,\ P_2P_3,\ P_2A_2,\ A_2P_3, P_2I_2,\ P_1A_1,\ P_1I_1,\ I_1A_3, I_1I_3, \cdots\}$. f_Q maps tree nodes to meta-paths, such as $f_Q(A_2) = PPA$. f_W maps edges to adjacency matrices, such as $f_W(P_2, A_2) = \hat{A}_{PA}$.

To shorten the search time effectively, we propose a search strategy inspired by the Monte Carlo tree search[30], which reduces the huge number of meta-paths to a constant scale and efficiently screens out suitable meta-paths from them.

The Monte Carlo tree search method involves four steps: Selection, expansion, simulation and back propagation.

Selection. The selection operation determines the moving direction of the next step by using the node score function:

$$score = \alpha_1 \frac{(1-\eta)+\delta}{n_i} + \alpha_2 \sqrt{\frac{\ln N_i}{n_i}}$$
$$\eta = f(\mathcal{P}_i) \in \Phi_{node} \tag{1}$$

where $\alpha = (\alpha_1, \alpha_2)$ is the hyperparameter, the larger the α_2 is, the more inclined to the breadth search, and the smaller the α_2 is, the more inclined to the depth search; f is the preliminary evaluation function of the meta-paths; η is the score of the meta-paths (such as the training accuracy); n_i is the simulation time of the child nodes of the i-th node of the meta-path tree, and N_i is the total simulation time of the i-th node, $\sqrt{\frac{\ln N_i}{n_i}}$ is used to control the search direction.

A preliminary evaluation method of meta-path is adopted to improve the selection efficiency. Assuming that the meta-path to be scored is $\mathcal{P}_i$, the corresponding characteristic matrix of the meta-path is $X^{\mathcal{P}_i}$ (the specific calculation method of $X^{\mathcal{P}_i}$ is introduced in Section 4.2). The node embedding obtained by using multi-layer perceptron (MLP) and a round of learning is $Pred^{\mathcal{P}_i} = MLP^{\mathcal{P}_i}(X^{\mathcal{P}_i})$. The prediction accuracy of the embedding is calculated by the preliminary evaluation score as

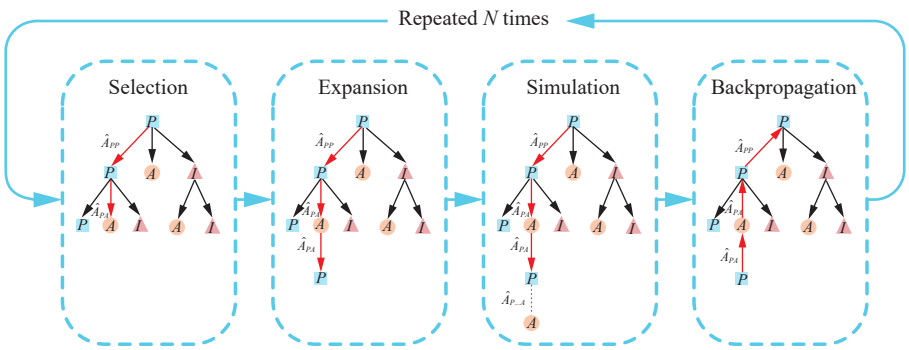


Fig. 3 Meta-path search strategy based on Monte Carlo tree search (Colored figures are available in the online version at https://link.springer.com/journal/11633)

$$f\left(\mathcal{P}_i\right) = Accuracy\left(Y, Pred^{\mathcal{P}_i}\right) \tag{2}$$

where Y is the label.

Expansion. The expansion operation adds a node to the meta-path tree after selection operation, calculates the corresponding meta-path score of the added node, and uses an expansion method to determine whether to expand a given node by storing one or more child nodes in memory. Commonly, there are two kinds of expansion methods. The first adds a single node per simulation time, and the added node is situated at the first position without being stored in memory during traversal. The second expands the tree to a certain depth, and add all the children of a node to the tree before starting a new search. To make the meta-path search strategy simple, efficient, and easy to implement, the first expansion method is adopted in Section 4.1.

Simulation. The simulation operation randomly selects the direction and time of node movement before the end of a finite time horizon. It can realize the random sampling of long-distance meta-path to determine whether there is valuable high-order semantic information.

The simulation operation often requires a trade-off between time and knowledge. Appropriately increasing the number of simulations and designing a simulation algorithm can enable us to find more long-range meta-paths and capture more valuable high-order semantic information. Because the data of nodes may be too large in the actual process, setting an accurate evaluation method will lead to slow movement, which will reduce the number of simulations that can be carried out in a unit time. Therefore, we use the random fast selection method to randomly move down from the extended node and fix the number of moves to realize the random simulation process.

To avoid insufficient meta-path search due to random simulation and computational budget constraints, the number of simulations can be set according to the network specifications. In complex multi-relational networks, the number of simulations should be appropriately increased, and the simulation length needs to be gradually extended, thereby achieving a balance between the computational budget and accuracy.

Back propagation. The back propagation operation adds the simulation results to the tree, and updates the average score and access times of the tree with δ generated by iteration. In this operation, the simulated score is returned to the last visited node (extended node) in the tree to all nodes on the root node path, and their average score and simulation times are updated. The parameter δ is calculated by

$$\delta = 1 - f\left(\mathcal{P}\right) \tag{3}$$

where $\mathcal{P}$ is the long-distance meta-path obtained by simulation, and f is the preliminary evaluation function of the meta-path. The use of back propagation can ensure that the selection strategy has a certain directionality, which is conducive to the subsequent sampling of meta-paths with richer semantic information. The above steps are repeated to obtain the simulation scores and diversity of the mobile nodes (meta-paths), sort them, and select the k nodes with the highest scores. The k meta-paths determined by the k nodes are appropriate meta-paths:

$$\begin{aligned} \Phi_{node}\left(k\right) &= sort\left(\Phi_{node}, \max, k\right) \\ \Phi_X\left(k\right) &= \left\{P_i \ \in \ f^{-1}\left(\Phi_{node}\left(k\right)\right)\right\}. \end{aligned} \tag{4}$$

Due to the high randomness of single-round meta-path search, it may result in insufficient sampling of meta-paths and prevent comprehensive evaluation. Multiple rounds of search are performed with resetting parameters at each round and expanding from the root node. Assuming that the total number of search rounds is t_{all}, the union of the score sets for each round of meta-paths is obtained as follows:

$$\Phi_{node} = \bigcup_{i=0}^{t_{all}} \Phi_{node}^{i}. \tag{5}$$

The nodes in Φ_{node} are sorted and the top k nodes with the highest scores are selected. Accordingly, the k meta-paths determined by these top nodes are treated as the appropriate meta-paths. The overall algorithm flow for meta-path search is shown in Algorithm 1.

Assuming the sizes of the node type set and the relation type set are both r and the maximum hop count is l, the computation complexity is $O\left(r^{2l}\right)$ when directly enumerating all meta-paths. For the MCTS method, l layers are required to be traversed from the root node to a leaf node during the selection and expansion phases. In this procedure, the evaluation values of r^2 child nodes are calculated for each layer, resulting in a complexity of $O\left(lr^2\right)$. The simulation phase involves at most l hop evaluations with complexity $O\left(l\right)$, while the back-propagation phase updates node information along the path with complexity $O\left(l\right)$. Consequently, the complexity of a single selection-backpropagation process is $O\left(lr^2\right)$. After K search iterations with each iteration consuming time t, the overall complexity of the MCTS meta-path search is $O\left(Ktlr^2\right)$.

Algorithm 1. Monte Carlo tree search (MCTS)

Input: Meta-path tree T_P, the number of search rounds t_{all}, single round search time t_{one}, hyper-parameters $\alpha = (\alpha_1, \alpha_2)$, the target number of meta-paths k

Output: Meta-paths sets Φ_X

1) **for** $i = 0$ to the number of search rounds t_{all} **do**
2) The starting node of the meta-path is c_0
3) **for** $j = 0$ to the number of tree expansions (or time t) **do**

4) $c = select\,(c, \alpha)$
5) $c = expend\,(c)$
6) $x = simulate\,(c)$
7) $backpropagate\,(c, x)$
8) **end for**
9) All node score Φ^i_{node}
10) **end for**
11) $\Phi_{node} = \bigcup_{i=0}^{t_{all}} \Phi^i_{node}$
12) $\Phi_{node}\,(k) = sort\,(\Phi_{node}, \max, k)$
13) $\Phi_X = \{\mathcal{P}_i \in f^{-1}(\Phi^{(k)}_{node})\}$
14) **return** Φ_X

4.2 Node aggregation

Through the meta-path search step in Section 4.1, we select the top k meta-paths among many meta-paths and generate the connection graph using these meta-paths, and perform the node aggregation step, so as to realize the information transmission between neighbor nodes. Commonly, methods such as average pooling, maximum pooling, splicing process, and attention mechanism are applied in the node aggregation step. The general formula for node aggregation is expressed as

$$h_{N(v_i)} = AGG\left(\left\{h_{v_j}, \forall v_j \in N_S\,(v_i)\right\}\right) \tag{6}$$

where $h_{N(v_i)}$ is the feature vector aggregated from the neighbors of node v_i, h_{v_j} represents the feature vector of node v_j, $N_S\,(v_i)$ represents the sampled direct neighbors of node v_i, and AGG is the aggregation function.

Algorithm 2. Heterogeneous graph neural network model

Input: Raw feature matrices $\{X^{c_i} : c_i \in \mathcal{A}\}$, the raw label matrix Y, meta-path sets $\Phi_{\mathcal{P}}$, hyper-parameters α, the number of search rounds t_{all}, single round search time t_{one}, the target number of meta-paths k

Output: Node classification results $Pred$ for target type c

1) Building a meta-path tree $\Phi_{\mathcal{P}} \to T_{\mathcal{P}}$
2) $\Phi_X = MCTS\,(T_{\mathcal{P}}, t_{all}, t_{one}, \alpha, k)$
3) **for** $\mathcal{P}_i \in \Phi_X$ **do**
4) $X^{\mathcal{P}_i} = \hat{A}_{cc_l} \cdots \hat{A}_{c_{(l-1)}c_l} X^{c_l}$
5) $H^{\mathcal{P}_i} = X^{P_i} \cdot W^{P_i}$
6) **end for**
7) $\Phi_X = \bigcup_{l=1}^{|\mathcal{A}|} \Phi_l,\ \Phi_l = \{\mathcal{P} = c \cdots c_l : c_l \in \mathcal{A}\}$
8) **for** $\Phi_l = \Phi_1, \Phi_2, \cdots, \Phi_{|\mathcal{A}|}$ **do**
9) $H_l = Transformer\left(\left\{H^{\mathcal{P}_i} : \mathcal{P}_i \in \Phi_l\right\}\right)$
10) **end for**
11) $H = Transformer\left(\{H_l : l = 1, \cdots, |\mathcal{A}|\}\right)$
12) $Pred = MLP\,(H)$
13) $\mathcal{L} = -\sum_i y_i \ln pred_i$
14) Do back-propagation and update network parameters
15) **return** $Pred$

The node aggregation module follows the SeHGNN, which calculates the final contribution weight of each adjacent node to the target by multiplying the adjacency matrix, and only needs to be executed once during the aggregation process. Assuming X^{c_l} is the original characteristic matrix of c_l type, the feature matrices of the meta-path $\mathcal{P}$ is described as

$$X^{\mathcal{P}} = \hat{A}_{cc_l} \cdots \hat{A}_{c_{(l-1)}c_l} X^{c_l} \tag{7}$$

where $\mathcal{P}$ denotes the met-path ($\mathcal{P} = cc_1 \cdots c_l$), $\hat{A}_{c_i c_{(i+1)}}$ is the normalized adjacency matrix between node types c_i and $c_{(i+1)}$.

Generally, the original characteristic matrix corresponding to different node types has different dimensions. Therefore, we use the MLP for feature projection to map node features to the same embedded space:

$$H^{\mathcal{P}} = X^{\mathcal{P}} \cdot W^{\mathcal{P}} \tag{8}$$

where $X^{\mathcal{P}} \in \mathbb{R}^{n \times d}$, $W^{\mathcal{P}} \in \mathbb{R}^{d \times f}$, and $W^{\mathcal{P}}$ is the parameter to be learned. Feature projection mitigates the issue of scattered weights in subsequent classification tasks caused by heterogeneous nodes residing in disparate dimensional spaces. Through the above steps, node aggregation is achieved by using meta-paths to get the embedding of nodes under different meta-paths. These embeddings contain the semantic and structural information of meta-paths. In Section 4.3, we make full use of this information through the semantic fusion module.

4.3 Semantic fusion

To integrate semantic feature vectors and generate the final embedding vector for each node efficiently, a new attention-based meta-path aggregation method is proposed, inspired by transformer[31] and SeHGNN, in this section. The method can capture rich semantic information between different meta-paths. The specific steps are as follows.

Semantic information varies with meta-paths. We hope to find the semantic relationships among meta-paths with the same aggregation object type, obtain richer and more comprehensive semantic information, and understand the importance of these meta-paths. Assuming that the set of meta-paths obtained after the search is Φ_X, it is decomposed according to the type of aggregation object nodes of the meta-paths as

$$\Phi_X = \bigcup_{l=1}^{|\mathcal{A}|} \Phi_l, \Phi_l = \{\mathcal{P} = c_0 \cdots c_l : c_l \in \mathcal{A}\}. \tag{9}$$

Taking a simple graph as an example (as shown in Fig. 4), the node type set is $\mathcal{A} = \{P, I, A\}$, and the selected meta-path set is $\Phi_X = \{PP, PI, PA, PIP, PII, PAPI, PPAA\}$. The set Φ_X is decomposed with (9) into three subsets as

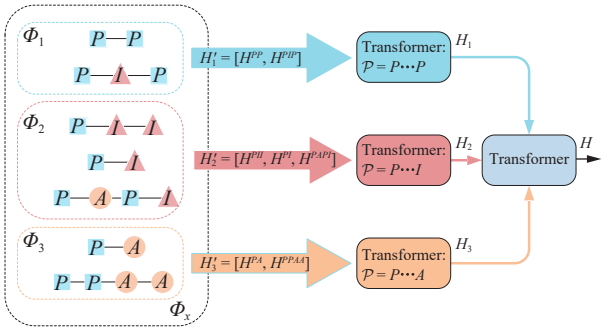


Fig. 4 Semantic fusion (Colored figures are available in the online version at https://link.springer.com/journal/11633)

$$\Phi_X = \Phi_1 \cup \Phi_2 \cup \Phi_3 \tag{10}$$

where $\Phi_1 = \{PP, PIP\}$, $\Phi_2 = \{PI, PII, PAPI\}$ and $\Phi_3 = \{PA, PPAA\}$.

After decomposition, the meta-paths in Φ_l have the same aggregation object type. The semantic relationship between these meta-paths with the same aggregated object type is found to learn the importance of these meta-paths. In this paper, transformer is adopted to learn the relationships as

$$H_l = Transformer\left(\left\{H^{\mathcal{P}_i} : \mathcal{P}_i \in \Phi_l\right\}\right) \tag{11}$$

where $H^{\mathcal{P}_i}$ is the node embedding feature generated by meta-path $\mathcal{P}_i$, and H_l is the node embedding feature under the meta-path subset Φ_l. The specific process of transformer is shown in Fig. 5.

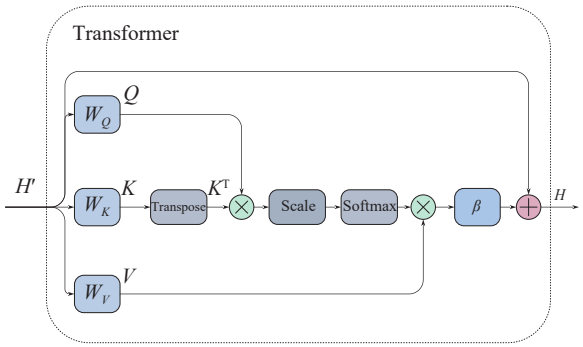


Fig. 5 Specific process of the transformer (Colored figures are available in the online version at https://link.springer.com/journal/11633)

The characteristic matrix $H^{\mathcal{P}_i}$ of all $\mathcal{P}_i$ in the meta-path subset is spliced with each other to obtain the characteristic matrix H_l' of the meta-path subset as

$$H_l' = \left[H^{\mathcal{P}_1}, H^{\mathcal{P}_2}, \cdots, H^{\mathcal{P}_m}\right] \tag{12}$$

where $H_l' \in \mathbb{R}^{n \times mf}$, $\mathcal{P}_i \in \Phi_l$, $H^{\mathcal{P}_i} \in \mathbb{R}^{n \times f}$. The corresponding query Q_l, keys K_l, and values matrices V_l are calculated as

$$Q_l = W_{Q_l} H_l',\ K_l = W_{K_l} H_l',\ V_l = W_{V_l} H_l' \tag{13}$$

where W_{Q_l}, W_{K_l}, W_{V_l} are the parameters to be learned. With Q_l, K_l and V_l, the dot product attention is determined as

$$Attention\left(Q_l, K_l, V_l\right) = \text{softmax}\left(\frac{Q_l K_l^{\mathrm{T}}}{\sqrt{mf}}\right) V_l \tag{14}$$

where $\frac{1}{\sqrt{mf}}$ is the scale factor used to prevent the calculated dot product from being too large. To compare the weight coefficient with each other of different meta-path, softmax is applied in the coefficient's calculation and normalization. The characterization ability of the network is improved through residual connection:

$$H_l = \beta_l Attention\left(Q_l, K_l, V_l\right) + H_l' \tag{15}$$

where β_l is the parameter to be learned.

The feature embedding $H_1, H_2, \cdots, H_{|\mathcal{A}|}$ of meta-path subsets $\Phi_1, \Phi_2, \cdots, \Phi_{|\mathcal{A}|}$, from the meta-path sets of different object types, are obtained after the above operations. To estimate the heterogeneous semantic relationships between meta-paths of different object types, find the importance of feature embedding, and assign different weight coefficients to them, we also use a transformer to perform calculations. The calculation process of the transformer is the same as (11), but the input is the node feature embedding of different meta-path subsets. Due to the significant differences in the object types and connection relationships of meta-path aggregation in different meta-path subsets, these differences are the main manifestations of heterogeneity in the graph. Therefore, this method can more effectively learn the heterogeneous semantic information between meta-path subsets, so as to obtain the complete node feature embedding of the graph:

$$H = Transformer\left(\{H_l : l = 1, \cdots, |\mathcal{A}|\}\right). \tag{16}$$

The calculation process of transformer is the same as above, and where $H \in \mathbb{R}^{n \times mf|\mathcal{A}|}$ is the embedding feature of complete graph nodes. The obtained node embedding H can be used for downstream task analysis. For example, when nodes are classified, the prediction result is $Pred = MLP(H)$.

5 Experiment

To verify the effectiveness of the proposed MCTS-HGNN framework, we evaluate it through downstream tasks, including node classification, node clustering and visualization in several heterogeneous graph datasets. Accordingly, a series of ablation experiments are conducted to demonstrate the performance of the designed individual modules. Parameter analysis is also utilized to describe the impact of these model parameters on performance.

5.1 Datasets

The experiments are conducted on real-world heterogeneous graph benchmark (HGB)[32]. The details of the three datasets are shown in Table 1.

DBLP is a bibliographic website for computer science journals and proceedings, where we extract a subset with four node types, such as author (A), paper (P), term (T) and venue (V), and six edge types. The node classification and clustering are performed on the author, which is labelled with four classes.

The IMDB is a website about movies and related information. We extract a subset of three node types: movie (M), director (D) and author (A). The node classification and clustering on movie are conducted with five labelled classes.

ACM is a citation network. We use a subset of the network with three types, containing paper (P), author (A) and subject (S). The target nodes in node classification and clustering are paper, which are with three labels.

5.2 Baselines

For comparison, some state-of-the-art baselines including homogeneous graph models (e.g., GCN, GraphSAGE and GAT) and heterogeneous graph models (e.g., HAN, RGCN, SimpleHGN, LMSPS, SeHGNN, HAE and HMSG) are selected in this section.

GCN[1] (2017) aggregates messages layer by layer from neighbors through the convolution of first-order spectral graphs to calculate the node representation.

GraphSAGE[20] (2017) generates embedded functions by sampling and aggregating features from the local neighborhood of nodes.

GAT[2] (2017) uses attention to calculate the importance of different neighbors.

HAN[7] (2019) learns node embedding by applying different attention in node and semantic levels.

RGCN[12] (2018) processes relational data through relation specific transformation and information transfer methods.

SimpleHGN[33] (2021) incorporates both edge type and node embeddings and designs a relation attention coefficient on the basis of GAT.

LMSPS[23] (2024) uses a progressive sampling strategy to search for meta paths, while MLP performs semantic fusion.

SeHGNN[17] (2023) calculates neighbor aggregation with a lightweight mean aggregator, and uses a single-layer structure with long meta-paths and a semantic fusion module.

HAE[33] (2023) combines meta-paths and metagraphs to learn heterogeneous semantics, and applies a self-attention mechanism to explore the interaction of nodes.

HMSG[34] (2023) decomposes the heterogeneous graph into multiple homogeneous and heterogeneous subgraphs based on meta-path, and learns the subgraphs by a self-attention mechanism.

5.3 Experimental settings

For the MCTS-HGNN model, node classification and node clustering tasks were employed for evaluation. The MCTS-based meta-path search algorithm utilized the validation accuracy of a simple MLP trained for 1 epoch as the initial meta-path score, with a computational budget set to 5. In the HGNN framework, the learning rate and weight decay were configured as 0.001 and 0, respectively. Feature projection was implemented through multi-layer pointwise 1D convolutions, and the Adam[35] optimizer was adopted for model training with detailed hyperparameters specified in Table 2. For baseline models, all configurations strictly follow their original publications using optimal parameters to ensure fair comparisons.

5.4 Node classification

Node classification is a commonly used downstream task for evaluating heterogeneous graph models. We conducted node classification experiments on the above three datasets to evaluate our model and baseline methods. The evaluation indicators are Micro-F1 score and Macro-F1 score. For node classification tasks, five training sets are randomly selected from the original graph set. Experiments are performed on these subsets whose sizes account for 20%, 40%, 60% and 80% of the original one′s. The average experimental result is reported as the evaluation indicator. The experimental results are shown in Table 3.

The experimental results show that the performance of the models used for heterogeneous graph algorithm is generally better than those used for homogeneous graph, proving that the model containing heterogeneous semantics performs better than the model with homogeneous semantics in heterogeneous graph. In the homogeneous graph model, GAT has the best classification result, and its average scores are significantly higher than those of GCN and GraphSAGE, with about 4%–5% performance improvement on DBLP, about 8%–9% performance

Table 1 The specifications of the dataset

Datasets	Nodes	Node types	Edges	Edges types	Targets	Classes
DBLP	26 128	4	239 566	6	Author	4
IMDB	21 420	3	86 642	6	Movie	5
ACM	10 942	4	547 872	9	Paper	3

Table 2 Hyperparameters

Datasets	Max hops	Feature projection	Hidden layer	α
DBLP	7	512	256	(1, 0.7)
IMDB	7	512	256	(1, 0.7)
ACM	6	256	512	(20, 0.7)

improvement on IMDB, and about 1%–2% improvement on ACM, indicating that GAT's graph attention also has a significant effect in heterogeneous graphs. In the heterogeneous graph model, SimpleHGN does not use the meta-path method, but uses residual, normalization and other technologies to enhance it on the basis of GAT. Compared with GAT, it has 4%–5% performance improvement in each dataset, and the performance of the model ranks the fourth in all baseline models. Its work shows that in some heterogeneous data sets, the model can achieve good classification effect without meta-path. Obviously, SeHGNN and our MCTS algorithm both perform better than other heterogeneous graph models. Compared with other heterogeneous models, the performance of SeHGNN has been improved by at least 1%, and some have been improved by up to 14%. This means that the precomputing based on lightweight mean aggregator for neighbor aggregation and semantic fusion greatly improve the model effectiveness. The number of meta-paths also has a significant impact on the model, and the performance of our proposed model is better than other models.

Additionally, the MCTS-HGNN is slight superior to the SeHGNN. For the DBLP and ACM datasets with relatively simpler structures, the performance improvement compared to SeHGNN is less pronounced. However, on the complex multi-label classification dataset IMDB, Macro-F1 score shows a 1% enhancement, with an improvement of approximately 2.2% under lower training set ratios. This might indicate that MCTS can effectively search the appropriate meta-path, and the designed semantic fusion module can capture richer heterogeneous semantic information.

5.5 Node clustering

Node clustering is another common downstream task for evaluating node embedding. We apply K-means for node clustering and set the number of clusters to the number of categories. The average normalized mutual information (NMI) and the adjusted rand index (ARI) are chosen as evaluation indexes. We randomly select 80% of the samples as the training set and repeat experiments five times. The average experimental results are reported in Table 4.

Table 3 Node classification results (%) compared to the baselines

			GCN	GraphSAGE	GAT	HAN	RGCN	SimpleHGN	LMSPS	SeHGNN	HAE	HMSG	MCTS-HGNN
DBLP	Micro-F1	20%	85.42	84.75	89.51	93.01	93.21	93.41	**94.64**	94.35	91.67	91.78	94.63
		40%	86.21	84.91	89.83	93.10	93.42	93.92	95.15	94.88	92.33	92.21	**95.18**
		60%	86.71	85.11	90.24	93.45	93.21	94.18	95.45	95.23	93.11	92.45	**95.49**
		80%	86.96	85.47	90.76	93.87	93.55	94.28	95.51	95.42	93.76	93.07	**95.66**
	Macro-F1	20%	85.11	84.43	89.42	92.33	91.51	93.22	**94.23**	93.88	91.57	91.65	94.18
		40%	85.95	84.97	89.61	92.41	92.06	93.75	**94.79**	94.49	93.02	92.02	**94.79**
		60%	86.54	85.14	90.11	92.78	92.17	93.97	95.10	94.88	93.16	92.51	**95.11**
		80%	86.87	85.33	90.69	93.02	92.34	94.04	95.17	95.07	93.67	92.91	**95.32**
IMDB	Micro-F1	20%	48.22	46.94	58.85	55.72	58.79	63.09	64.73	65.80	54.04	61.04	**66.25**
		40%	49.31	47.24	59.04	56.92	59.10	63.42	66.76	67.56	57.01	61.73	**68.25**
		60%	50.66	47.33	59.53	59.01	59.73	64.51	67.87	69.08	60.30	62.14	**69.35**
		80%	51.81	47.85	59.71	60.71	59.82	65.21	68.57	69.77	62.77	62.26	**69.85**
	Macro-F1	20%	47.14	46.63	58.43	52.11	58.51	62.04	63.18	62.13	54.51	60.53	**64.46**
		40%	48.17	46.87	58.87	53.73	58.76	62.36	65.17	64.53	57.09	61.43	**66.60**
		60%	49.26	47.11	59.42	54.34	59.64	63.04	66.24	66.53	60.62	61.77	**67.55**
		80%	50.55	47.65	59.68	56.57	59.77	63.50	66.91	67.28	62.81	61.96	**68.19**
ACM	Micro-F1	20%	86.51	85.87	88.74	89.03	91.33	92.03	**93.33**	92.48	89.64	91.34	92.89
		40%	86.87	86.22	88.91	89.34	91.77	92.57	**93.65**	93.37	91.22	91.46	93.43
		60%	87.34	86.41	89.21	89.21	92.04	93.10	**94.10**	93.62	91.70	91.68	93.81
		80%	87.94	86.65	89.42	89.73	92.41	93.37	94.04	93.97	92.83	91.72	**94.32**
	Macro-F1	20%	86.62	85.71	88.65	88.91	89.31	92.28	**93.41**	92.58	89.71	91.35	92.95
		40%	86.94	85.83	88.86	89.20	89.62	92.71	**93.74**	93.45	91.15	91.66	93.50
		60%	87.28	86.02	88.95	89.37	89.94	93.01	**94.17**	93.69	91.68	91.70	93.87
		80%	88.11	87.10	89.39	89.67	90.23	93.43	94.12	94.04	92.71	91.78	**94.37**

Table 4 Node clustering results (%) compared to the baselines

		GCN	GraphSAGE	GAT	HAN	RGCN	SimpleHGN	LMSPS	SeHGNN	HAE	HMSG	MCTS-HGNN
DBLP	NMI	67.41	65.22	68.31	79.76	80.34	84.32	85.30	85.02	84.21	84.34	**85.62**
	ARI	71.61	68.21	72.44	86.24	85.33	88.43	90.13	89.11	88.13	87.06	**90.37**
ACM	NMI	66.93	64.71	67.11	75.21	76.05	80.36	81.20	81.34	80.12	80.17	**81.86**
	ARI	69.14	66.81	73.31	81.03	81.06	85.34	86.12	86.28	84.64	85.07	**86.72**

The results show that there is a significant performance gap between the homogeneous graph model and the heterogeneous graph model, and the node clustering performance of heterogeneous graph model is at least 7% higher than that of homogeneous graph model, which also reflects that the model designed for heterogeneous graph can better capture the heterogeneous characteristics in the graph. It is observed that the performance of MCTS-HGNN surpasses that of the other baselines, with a minimum improvement of 0.4% compared with other heterogeneous graph models, and some can achieve a 5% improvement. Consequently, our proposed meta-path search algorithm based on MCTS can effectively search the appropriate meta-path and determine the central node more accurately for clustering.

5.6 Visualization

To intuitively display the clustering effect, visual comparisons are conducted among GraphSAGE, GAT, HAN, LMSPS, SeHGNN, MCTS-HGNN on the DBLP datasets, with the number of clusters set to be consistent with the number of real categories. t-SNE is adopted to visualize the embeddings, as shown in Fig. 6.

From Fig. 6, we can observe that homogeneous graph methods fail to perform well. The works from different research areas, by applying GraphSAGE, are mixed with each other. HAN performs slightly better than GAT. However, the boundary between the clusters is still not very clear. As expected, MCTS-HGNN obviously excels other methods with correctly identified clusters and clear boundaries. This shows that our model has stronger cohesion and effectiveness in heterogeneous networks.

5.7 Parameters sensitivity

To understand the importance of the meta-paths, a detailed parameter analysis of the proposed method is provided, containing the number of meta-paths, the feature projection embedding dimension size, the hidden layer dimension size and the search iterations.

The results of node classification with the different number of meta-paths are reported in Fig. 7. From Fig. 7, it is found that the performance of the model is significantly improved when the number of meta-paths is greater than 5. The model needs enough meta-paths to capture the heterogeneous semantic information. However, the model accuracy stops increasing when the number

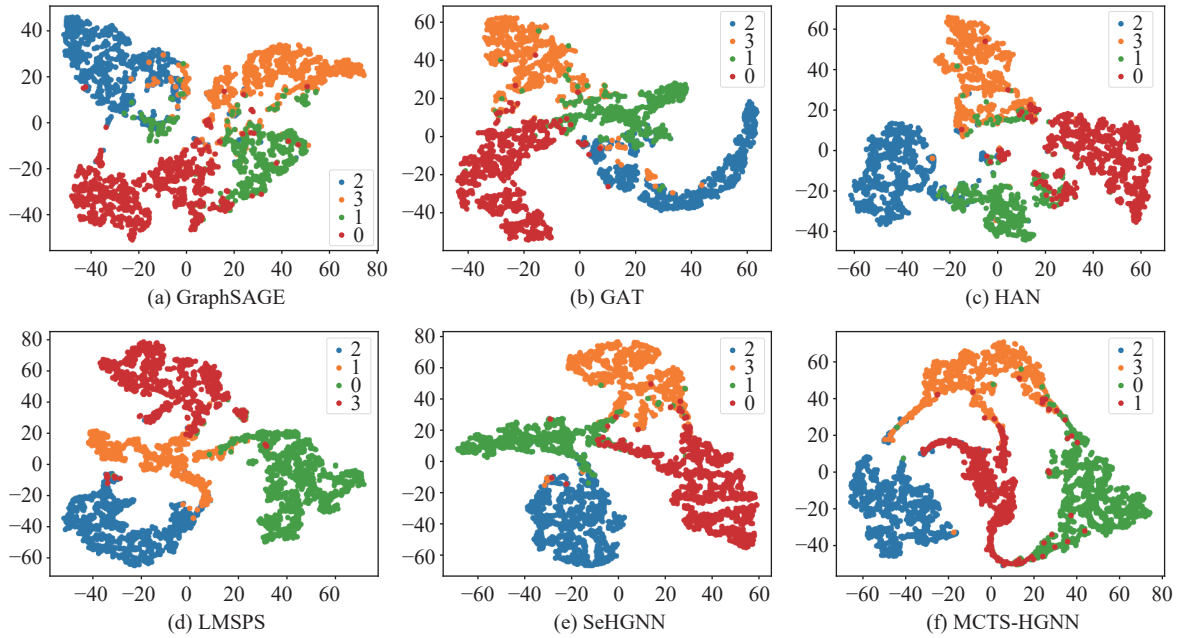


Fig. 6 Clustering visualization on DBLP (Colored figures are available in the online version at https://link.springer.com/journal/11633)

of meta-paths reaches 20. Although meta-paths play a leading role in the model performance, the continuous increase of meta-paths will not provide additional semantic information after the number touching a critical value.

The node classification experiments are conducted on three datasets to evaluate the impact of the feature projection embedding dimension size on model performance with other fixed parameters. The experiment results are shown in Fig. 8. It is found that the accuracy of node classification on the three datasets gradually increases with the embedding dimension. A higher embedding dimension can capture more heterogeneous semantic information.

The experimental results of the hidden layer dimension size with three datasets are shown in Fig. 9. It is observed that the performance first increases and then remains stable as the hidden layer dimension increases. A higher hidden layer dimension can capture node attributes and structural information more comprehensively. However, the parameters and complexity of the model also increase with the hidden layer dimension.

Fig. 10 describes the experimental accuracy results under the different numbers of search iterations t_{all}. As the number of search iterations increases, the accuracy and stability of the model are gradually improved, but this also leads to a prolonged search time. Excessive search iterations beyond a certain threshold may increase the likelihood of misestimating certain meta-paths, thereby introducing harmful semantic information and resulting in performance degradation.

Table 5 shows the impact of varying the maximum hop numbers on search time and model performance. As shown in Table 5, both the performance and search time of MCTS-HGNN increase with the higher maximum hop numbers. However, the excessively large hop numbers

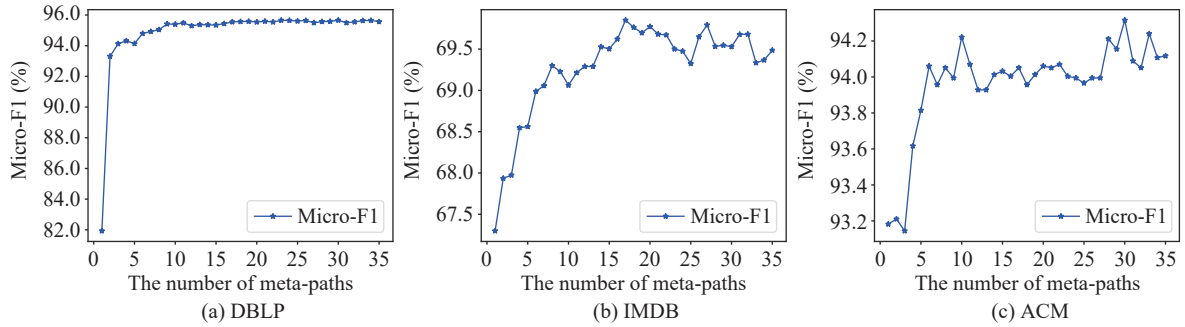


Fig. 7 Node classification results with a different number of meta-paths

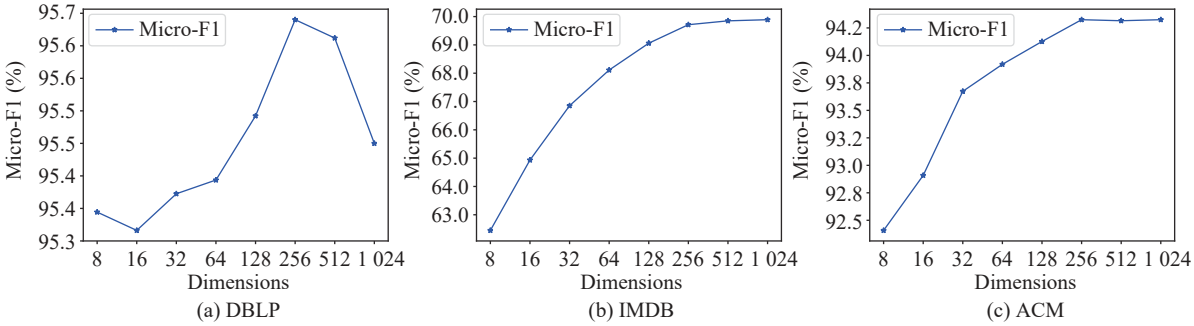


Fig. 8 Node classification results with different feature projection embedding dimension sizes

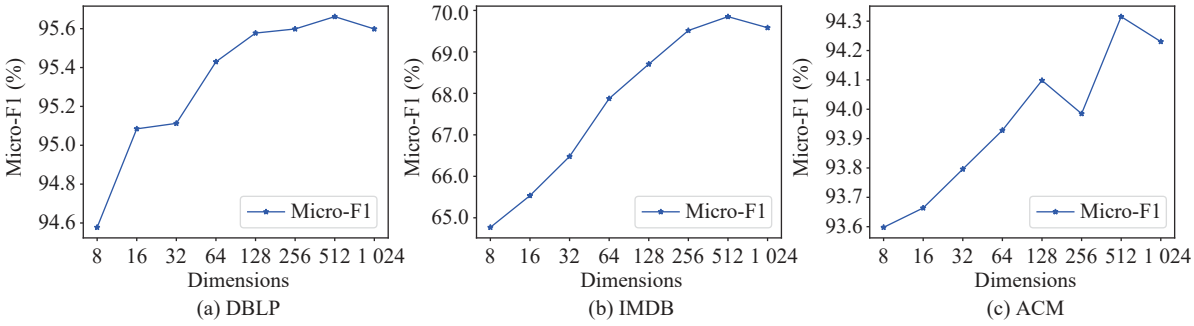


Fig. 9 Node classification results with different hidden dimension sizes

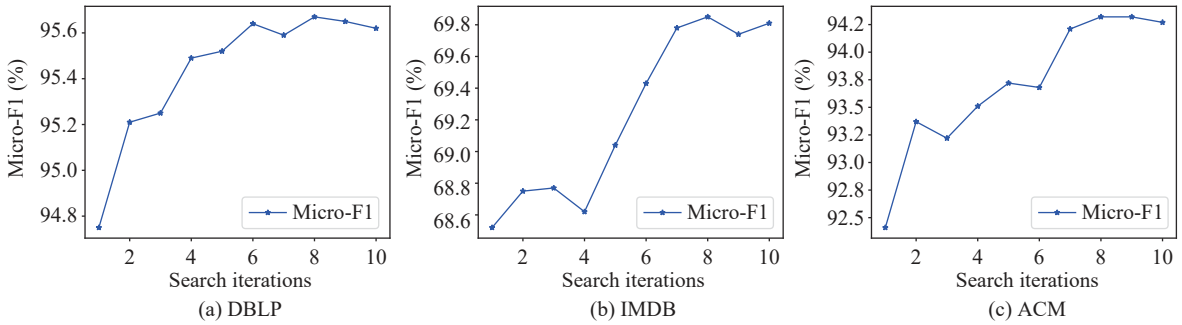


Fig. 10 Node classification results with different search iterations

may lead to a combinatorial explosion of meta-paths, increasing the search complexity and the risks of introducing harmful semantic information.

Increasing search duration enhances model performance, but also exhibits diminishing marginal returns and incurs a significant rise in computational costs. A desirable hop number should be selected according to specific scenarios. For environments with high real-time demands, prioritizing lower hop numbers (2–4) is recommended, trading marginal performance degradation for efficiency gains. Conversely, for performance-critical scenarios, the higher hop numbers (6–7) may be adopted, though the stability of performance gains must be rigorously validated. The heterogeneity across datasets leads to variability in the optimal hop numbers, with complex heterogeneous graphs potentially requiring higher hop meta-paths to capture intricate relationships.

5.8 Ablation study

We carry out node classification experiments on $\text{HGNN}_{\text{Full}}$, $\text{HGNN}_{\text{MCTS}}$, HGNN_{SF}, and $\text{HGNN}_{\text{MCTS,SF}}$ respectively, to describe the impact of the proposed meta-path search algorithm based on MCTS and semantic fusion module on the performance of the model, as shown in Table 6.

$\text{HGNN}_{\text{Full}}$ uses all meta-paths with fixed hops for lightweight mean node aggregation, and uses the MLP for simple semantic fusion. After the meta-path search module (MCTS) is added, the performance of the model is improved by 0.3%–0.7%, the performance of the model is improved by about 0.3%–0.5% after adding the semantic fusion (SF) module, and the performance of the model is improved by about 0.4%–0.7% after the meta-path search module and semantic aggregation module are added. The experimental results show that deleting the meta-path search algorithm or semantic fusion module of MCTS leads to a certain degree of degradation of the model performance, which again verifies the effectiveness and necessity of the proposed meta-path search algorithm and semantic fusion module based on MCTS.

Table 5 Experiments to analyze the performance of MCTS-HGNN under different maximum hops

Max hop	DBLP			IMDB			ACM		
	Time (s)	Micro (%)	Macro (%)	Time (s)	Micro (%)	Macro (%)	Time (s)	Micro (%)	Macro (%)
2	3.13	94.61	94.13	3.63	68.23	66.42	2.68	93.44	93.49
3	6.68	95.53	95.17	7.85	68.79	66.96	7.00	93.63	93.69
4	11.35	95.60	95.23	14.56	69.39	67.67	10.62	93.96	94.01
5	18.94	95.58	95.24	23.62	69.32	67.65	17.63	94.08	94.14
6	26.09	95.48	95.11	35.51	69.85	68.10	26.11	94.19	94.25
7	37.30	95.66	95.32	52.07	69.33	97.51	36.29	94.32	94.37

Table 6 Ablation study

Models	DBLP		IMDB		ACM	
	Micro-F1 (%)	Macro-F1 (%)	Micro-F1 (%)	Macro-F1 (%)	Micro-F1 (%)	Macro-F1 (%)
$\text{HGNN}_{\text{Full}}$	95.25	95.01	69.31	67.65	93.56	93.73
$\text{HGNN}_{\text{MCTS}}$	95.53	95.30	69.77	68.13	94.09	94.15
HGNN_{SF}	95.58	95.23	69.69	68.00	94.06	94.13
$\text{HGNN}_{\text{MCTS,SF}}$	95.66	95.32	69.85	68.19	94.32	94.37

6 Conclusions

In this paper, an improved Monte Carlo tree search-based heterogeneous graph neural network model has been developed for representation learning. First, the Monte Carlo tree search strategy has been proposed to autonomously explore meta-paths, addressing the challenge of selecting appropriate meta-paths. Second, the meta-path set decomposition and transformer have been employed to fully capture the semantic information embedded in meta-paths. Results from ablation studies have validated the effectiveness of our approach and demonstrated its capability to effectively learn complex semantic features within heterogeneous graphs. Experiments have shown that the developed model is superior to the current advanced baseline in node classification, node clustering, and visualization tasks. In future work, leveraging intermediate meta-based feature extraction could further enhance the model by fully exploiting long-distance heterogeneous information encoded in meta-paths.

Acknowledgements

This work was supported by the Basic and Applied Basic Research Foundation of Guangdong Province, China (Nos. 2023A1515240034 and 2024A1515240069), National Natural Science Foundation of China (Nos. 62173099 and 62303492), Natural Science Foundation of Hunan Province, China (No. 2025JJ40056), Postdoctoral Fellowship Program of CPSF, China (No. GZC20251165), China Postdoctoral Science Foundation (No. 2024M763699).

Declarations of conflict of interest

The authors declared that they have no conflicts of interest to this work.

References

[1] T. N. Kipf, M. Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, Toulon, France, 2017.

[2] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio. Graph attention networks. In *Proceedings of the 6th International Conference on Learning Representations*, Vancouver, Canada, 2018.

[3] G. Cui, J. Zhou, C. Yang, Z. Liu. Adaptive graph encoder for attributed graph embedding. In *Proceedings of the 26th International Conference on Knowledge Discovery & Data Mining*, New York, USA, pp. 976–985, 2020. DOI: 10.1145/3394486.3403140.

[4] Z. Guo, Z. Wu, T. Xiao, C. Aggarwal, H. Liu, S. Wang. Counterfactual learning on graphs: A survey. *Machine Intelligence Research*, vol. 22, no. 1, pp. 17–59, 2025. DOI: 10.1007/s11633-024-1519-z.

[5] Z. Chen, B. Zhang, C. Du, W. Meng, A. Meng. A novel dynamic spatio-temporal graph convolutional network for wind speed interval prediction. *Energy*, vol. 294, Article number 130930, 2024. DOI: 10.1016/j.energy.2024.130930.

[6] C. Shi, Y. Li, J. Zhang, Y. Sun, P. S. Yu. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 1, pp. 17–37, 2017. DOI: 10.1109/TKDE.2016.2598561.

[7] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu. Heterogeneous graph attention network. In *Proceedings of World Wide Web Conference*, San Francisco, USA, pp. 2022–2032, 2019. DOI: 10.1145/3308558.3313562.

[8] Q. Mao, Z. Liu, C. Liu, J. Sun. HINormer: Representation learning on heterogeneous information networks with graph transformer. In *Proceedings of Web Conference*, Austin, USA, pp. 599–610, 2023. DOI: 10.1145/3543507.3583493.

[9] H. Ji, X. Wang, C. Shi, B. Wang, P. S. Yu. Heterogeneous graph propagation network. *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 521–532, 2023. DOI: 10.1109/TKDE.2021.3079239.

[10] X. Wang, D. Bo, C. Shi, S. Fan, Y. Ye, P. S. Yu. A survey on heterogeneous graph embedding: Methods, techniques, applications and sources. *IEEE Transactions on Big Data*, vol. 9, no. 2, pp. 415–436, 2023. DOI: 10.1109/TBDATA.2022.3177455.

[11] Y. Sun, J. Han, X. Yan, P. S. Yu, T. Wu. PathSim: Meta path-based top-K similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, vol. 4, no. 11, pp. 992–1003, 2011. DOI: 10.14778/3402707.3402736.

[12] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling. Modeling relational data with graph convolutional networks. In *Proceedings of the 15th International Conference on the Semantic Web*, Heraklion, Greece, pp. 593–607, 2018. DOI: 10.1007/978-3-319-93417-4_38.

[13] H. Hong, H. Guo, Y. Lin, X. Yang, Z. Li, J. Ye. An attention-based graph neural network for heterogeneous structural learning. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*, New York, USA, pp. 4132–4139, 2020. DOI: 10.1609/AAAI.V34I04.5833.

[14] T. Wei, T. W. S. Chow, J. Ma, M. Zhao. ExpGCN: Review-aware graph convolution network for explainable recommendation. *Neural Networks*, vol. 157, pp. 202–215, 2023. DOI: 10.1016/J.NEUNET.2022.10.014.

[15] L. Yu, L. Sun, B. Du, C. Liu, W. Lv, H. Xiong. Heterogeneous graph representation learning with relation awareness. *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5935–5947, 2023. DOI: 10.1109/TKDE.2022.3160208.

[16] C. Zhang, D. Song, C. Huang, A. Swami, N. V. Chawla. Heterogeneous graph neural network. In *Proceedings of the 25th International Conference on Knowledge Discovery & Data Mining*, Anchorage, USA, pp. 793–803, 2019. DOI: 10.1145/3292500.3330961.

[17] X. Yang, M. Yan, S. Pan, X. Ye, D. Fan. Simple and efficient heterogeneous graph neural network. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence*, Washington DC, USA, pp. 10816–10824, 2023. DOI: 10.1609/AAAI.V37I9.26283.

[18] W. Ning, R. Cheng, J. Shen, N. A. H. Haldar, B. Kao, X. Yan, N. Huo, W. K. Lam, T. Li, B. Tang. Automatic meta-path discovery for effective graph-based recommendation. In *Proceedings of the 31st International Conference on Information & Knowledge Management*, Atlanta, USA, pp. 1563–1572, 2022. DOI: 10.1145/3511808.

3557244.

[19] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, G. Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009. DOI: 10.1109/TNN.2008.2005605.

[20] W. L. Hamilton, Z. Ying, J. Leskovec. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Long Beach, USA, pp. 1024–1034, 2017.

[21] D. Lv, Z. Xu, J. Zhang, Y. Wang, F. Dong. Imbalanced node classification with Graph Neural Networks: A unified approach leveraging homophily and label information. *Applied Soft Computing*, vol. 149, Article number 110985, 2023. DOI: 10.1016/j.asoc.2023.110985.

[22] Y. Xiong, Z. Hu, C. Su, S. M. Cai, T. Zhou. Vital node identification in complex networks based on autoencoder and graph neural network. *Applied Soft Computing*, vol. 163, Article number 111895, 2024. DOI: 10.1016/j.asoc.2024.111895.

[23] C. Li, Z. Guo, Q. He, K. He. Long-range meta-path search on large-scale heterogeneous graphs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, Vancouver, Canada, Article number 1404, 2024.

[24] S. Ma, J. W. Liu, X. Zuo. Self-supervised learning for heterogeneous graph via structure information based on Meta-path. *Applied Soft Computing*, vol. 143, Article number 110388, 2023. DOI: 10.1016/j.asoc.2023.110388.

[25] B. Shi, T. Weninger. Mining interesting meta-paths from complex heterogeneous information networks. In *Proceedings of IEEE International Conference on Data Mining Workshop*, Shenzhen, China, pp. 488–495, 2014. DOI: 10.1109/ICDMW.2014.25.

[26] C. Meng, R. Cheng, S. Maniu, P. Senellart, W. Zhang. Discovering meta-paths in large heterogeneous information networks. In *Proceedings of the 24th International Conference on World Wide Web*, Florence, Italy, pp. 754–764, 2015. DOI: 10.1145/2736277.2741123.

[27] G. Wan, B. Du, S. Pan, G. Haffari. Reinforcement learning based meta-path discovery in large-scale heterogeneous information networks. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*, New York, USA, pp. 6094–6101, 2020. DOI: 10.1609/AAAI.V34I04.6073.

[28] Z. Han, F. Xu, J. Shi, Y. Shang, H. Ma, P. Hui, Y. Li. Genetic meta-structure search for recommendation on heterogeneous information network. In *Proceedings of the 29th International Conference on Information & Knowledge Management*, pp. 455–464, 2020. DOI: 10.1145/3340531.3412015.

[29] Y. Sun, J. Han. Mining heterogeneous information networks: A structural analysis approach. *ACM SIGKDD Explorations Newsletter*, vol. 14, no. 2, pp. 20–28, 2012. DOI: 10.1145/2481244.2481248.

[30] J. Kuipers, A. Plaat, J. A. M. Vermaseren, H. J. van den Herik. Improving multivariate Horner schemes with Monte Carlo tree search. *Computer Physics Communications*, vol. 184, no. 11, pp. 2391–2395, 2013. DOI: 10.1016/J.CPC.2013.05.008.

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, Ł. Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Long Beach, USA, pp. 5998–6008, 2017.

[32] Q. Lv, M. Ding, Q. Liu, Y. Chen, W. Feng, S. He, C. Zhou, J. Jiang, Y. Dong, J. Tang. Are we really making much progress?: Revisiting, benchmarking and refining heterogeneous graph neural networks. In *Proceedings of the 27th Conference on Knowledge Discovery & Data Mining*, Singapore, pp. 1150–1160, 2021. DOI: 10.1145/3447548.3467350.

[33] J. Li, H. Peng, Y. Cao, Y. Dou, H. Zhang, P. S. Yu, L. He. Higher-order attribute-enhancing heterogeneous graph neural networks. *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 560–574, 2023. DOI: 10.1109/TKDE.2021.3074654.

[34] M. Guan, X. Cai, J. Shang, F. Hao, D. Liu, X. Jiao, W. Ni. HMSG: Heterogeneous graph neural network based on Meta-path SubGraph learning. *Knowledge-Based Systems*, vol. 279, Article number 110930, 2023. DOI: 10.1016/J.KNOSYS.2023.110930.

[35] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, San Diego, USA, 2015.



Bin Zhang received the B. Sc. degree in electronic and information engineering from Xiangtan University, China in 2007, and the M. Sc. and Ph. D. degrees in control science and engineering from Central South University, China in 2010 and 2016, respectively. In 2016, she was a visiting internship student with the School of Electrical and Electronic Engineering, Adelaide University, Australia, for three months. Now, she is an associate professor with the School of Automation, Guangdong University of Technology, China.

Her research interests include smart grid technologies, process modeling, networked control systems, and optimal control.

E-mail: zhangbin_csu309@163.com

ORCID iD: 0000-0002-6743-7392



Zhipeng Cheng received the B. Sc. degree in mathematics and applied mathematics from Hubei University of Science and Technology, China in 2022. He is currently a master student in control science and engineering with the Guangdong University of Technology, China.

His research interest is graph neural networks.

E-mail: 2112204095@mail2.gdut.edu.cn



Chenglong Du received the B. Sc. degree in automation from Harbin Institute of Technology, China in 2016, the Ph. D. degree in control science and engineering from Central South University, China in 2022. From December 2019 to December 2021, he was a visiting Ph. D. student with the Department of Mechanical Engineering, University of Victoria, Canada. From August 2022 to June 2024, he was a postdoctoral fellow with the School of Automation, Central South University, China. Now, he is an associate professor with the School of Automation, Central South University, China.

His research interests include control theory, distributed systems and optimization, deep reinforcement learning, cyber–physical systems, and mechatronics.

E-mail: chenglong_du@csu.edu.cn (Corresponding author)

ORCID iD: 0000-0002-7879-7051



Zhengganzhe Chen received the B. Sc. degree in automation from Southwest University of Science and Technology, China in 2019, the Ph. D. degree in control science and engineering from Guangdong University of Technology, China in 2024. Now, he is a postdoctoral fellow with the School of Automation, Central South University, China.

His research interests include data analytics in smart grid and graph network-based energy forecasting.

E-mail: CZGZ_97@outlook.com



Chaoyang Chen received the Ph. D. degree in control science and engineering from the Huazhong University of Science and Technology, China in 2014. From 2015 to 2017, he was a post-doctoral with the School of Information Science and Engineering, Central South University, China. From 2018 to 2019, he was a visiting researcher with the Center for Polymer Studies and the Department of Physics, Boston University, USA. He is currently a professor with the School of Information and Electrical Engineering, the dean of the School of Information and Electrical Engineering and dean of the Hunan Provincial Key Laboratory of Intelligent Control and Maintenance of Complex Systems, Hunan University of Science and Technology, China. He is the recipient of the National Key Research and Development Program of China and the National Natural Science Foundation-Outstanding Youth Foundation in China, Vice President of the China Energy Society, and the IEEE Senior Member.

His research interests include networked control systems, complex networks, multiagent systems, and robust control, as well as their related applications.

E-mail: ouzk@163.com



Weihua Gui received the B. Sc. and M. Sc. degrees in control science and engineering from Central South University, China in 1976 and 1981, respectively. From 1986 to 1988, he was a visiting scholar with the University Duisburg-Essen, Germany. He has been a full professor with the School of Information Science and Engineering, Central South University, China since 1991.

His research interests include modeling and optimal control of complex industrial processes, distributed robust control, and fault diagnosis.

E-mail: gwh@csu.edu.cn